

DOC-07

VIXOR ENGINEERING BIBLE

كتاب المعايير الهندسية

الوثيقة الرسمية لمعايير وأساليب العمل الهندسية في فيكسور

جدول المحتويات

سير العمل في جيت (Git Workflow)

استراتيجية الفروع (Branch Strategy)

اتفاقية الالتزامات (Commit Convention)

طلبات السحب (Pull Requests)

مراجعة الكود (Code Review)

الاختبار (Testing)

التكامل والنشر المستمر (CI/CD)

عملية الإصدار (Release Process)

إدارة الإصدارات (Versioning)

قواعد التوثيق (Documentation Rules)

قواعد جودة الكود (Code Quality Rules)

قواعد الاعتماديات (Dependency Rules)

قائمة مراجعة المراجعة (Review Checklist)

سير العمل في جيت (Git Workflow)

يعتمد مشروع فيكسور بالكامل على نظام التحكم بالإصدارات **Git** كمصدر وحيد للحقيقة (Single Source of Truth). المستضاف على `github.com/kam65624-cmd/vixor-APP.git` ويُدار من خلال فرع `main` كفرع أساسي للاستقرار. عمليات النشر تتم تلقائيًا من هذا الفرع عبر **Vercel**، مما يعني أن أي كود يُدمج في `main` ينتقل مباشرة إلى بيئة الإنتاج بعد اجتياز عمليات الاختبار. يتبع فريق فيكسور نموذج سير عمل مبني على الفرع الرئيسي (Main-based Workflow) حيث يكون `main` دائمًا في حالة قياسية. تطوير الميزات الجديدة في فروع منفصلة تُدمج عبر طلبات السحب (Pull Requests) بعد المراجعة. هذا النهج يضمن أن بيئة الإنتاج تكون دائمًا مستقرة وسوى الكود المُراجع والمُختبر، ويقلل من مخاطر الأعطال ويسهل التتبع والرجوع عند الحاجة.

⚠ قاعدة ذهبية

يُمنع منعًا باتًا الدفع المباشر (Direct Push) إلى فرع `main`. جميع التغييرات يجب أن تمر عبر طلب سحب مع مراجعة كود واحدة على الأقل من فريق آخر.

المستودع

github.com/kam65624-cmd/vixor-APP.git — الفرع الافتراضي: main

مدير الحزم

pnpm الإصدار 9.15.0 مع Node.js >= 20.0.0 كحد أدنى

تتضمن أدوات المشروع الأساسية ثلاثة أوامر مُهيكلية: `pnpm dev` لتشغيل خادم التطوير عبر Vite، و `pnpm build` لتنفيذ عملية البناء. تشمل تنفيذ `vite build` متبوعًا بسكريبت `fix-vercel-bundle.mjs` لمعالجة مشاكل حزم النشر على Vercel، و `pnpm format` لتنسيق الكود عبر Prettier. لتشغيل فحص ESLint على المجلدات `src/` و `server/`، إضافة إلى `pnpm format` لتنسيق الكود عبر Prettier.

استراتيجية الفروع (Branch Strategy)

تتبع فيكسور استراتيجية فروع مبسطة وفعالة تركز على فرع واحد للاستقرار مع فروع مؤقتة للتطوير. الفرع `main` هو الفرع الوحيد الذي يمثل حالة التطبيق الجاهزة للإنتاج في أي لحظة. جميع الفروع الأخرى هي فروع مؤقتة تُحذف بعد دمجها لضمان نظافة المستودع.

الفرع الرئيسي — main

يحتوي `main` دائمًا على كود مستقر ومُختبر. كل التزام (Commit) في هذا الفرع يجب أن يمر عبر طلب سحب مُراجع. النشر إلى Vercel من هذا الفرع فقط، وعند دمج أي طلب سحب يبدأ Vercel عملية البناء فورًا.

فروع الميزات — */feature

كل ميزة جديدة أو تعديل يتم في فرع منفصل يُنشأ من `main`. يجب أن يحتوي اسم الفرع على وصف واضح ومختصر باللغة الإنجليزية. أن يكون الفرع مركّزًا على مهمة واحدة فقط لتسهيل المراجعة وتقليل التعارضات (Merge Conflicts).

نوع الفرع	الصيغة	مثال	الوصف
ميزة	feature/*	feature/add-stop-loss	تطوير ميزة جديدة بالكامل
إصلاح	fix/*	fix/auth-token-expiry	إصلاح خطأ محدد في الإنتاج أو التطوير
تحسين	refactor/*	refactor/query-cache-layer	إعادة هيكلة الكود بدون تغيير السلوك
وثائق	docs/*	docs/api-endpoints	تحديث أو إضافة وثائق
صيانة	chore/*	chore/update-deps	مهام صيانة وتحديثات بنية تحتية

🔥 دورة حياة الفرع

يُنشأ الفرع من main → يُطوّر ويُختبر → يُرسل طلب سحب → يُراجع ويُوافق عليه → يُدمج في main → يُحذف الفرع المؤقت.

03

اتفاقية الالتزامات (Commit Convention)

رغم عدم وجود تكوين صريح لاتفاقية الالتزامات في إعدادات المشروع الحالية، يُوصى بشدة باعتماد نمط **Conventional Commits** رسمي لفريق فيكسور. هذا النمط يوفر تاريخًا واضحًا ومنظمًا للتعديلات، ويسهّل إنشاء سجلات التغييرات (Changelogs) تلقائيًا أفضل مع أدوات النشر المستمر.

يعتمد النمط على صيغة موحدة تتكون من نوع الالتزام يتبعه نقطتان ثم وصف مختصر باللغة الإنجليزية. الوصف يجب أن يكون فـالمباشر (Imperative Mood) وألا يتجاوز 72 حرفًا. يمكن إضافة نص تفصيلي بعد سطر فارغ، وإضافة تذييل (Footer) لربط الالتزامات بطلبات سحب محددة.

```
feat(ding): add trailing stop-loss order type
fix(auth): resolve token refresh race condition
refactor(api): extract security middleware to _security.ts
chore(deps): update environment setup instructions
chore(deps): upgrade React to 19.2.0
```

أنواع الالتزامات المعتمدة

النوع	الاستخدام	يرفع الإصدار
feat	ميزة جديدة للمستخدم النهائي	PATCH → MINOR
fix	إصلاح خطأ في سلوك موجود	PATCH
refactor	إعادة هيكلة بدون تغيير السلوك	لا يرفع
docs	تغييرات في التوثيق فقط	لا يرفع
chore	مهام صيانة وبنية تحتية	لا يرفع
perf	تحسين في الأداء	PATCH
style	تنسيق وفواصل (لا منطق)	لا يرفع

طلبات السحب (Pull Requests)

طلبات السحب (Pull Requests) هي البوابة الوحيدة لدمج الكود في فرع `main`. كل طلب سحب يمثل وحدة تغيير مُمْتَحِنَة. يُشترط أن يحتوي كل طلب على وصف واضح يشرح السياق والتعديلات والاختبارات المُجرَاة، حتى يكون المراجع قادرًا على التغيير بسرعة وفعالية.

هيكل طلب السحب

يجب أن يتبع كل طلب سحب قالبًا موحدًا يضمن اكتمال المعلومات المطلوبة. يبدأ بعنوان واضح يعكس نوع التغيير والمكون المتأثر. يشرح لماذا تم هذا التعديل (السياق والمشكلة)، ثم قسم يفصل ما تم تغييره بالتحديد، وأخيرًا قسم للاختبارات وقائمة المراجعة (st) يؤكد المطور تنفيذها.

📋 قالب طلب السحب المطلوب

العنوان: type(scope): وصف مختصر

الوصف: شرح المشكلة والحل المقترح

التغييرات: قائمة بالملفات والوحدات المتأثرة

الاختبارات: وصف الاختبارات المُضافة أو المُعدّلة

قائمة المراجعة: تأكيد تنفيذ جميع النقاط قبل الطلب

شروط القبول

- يجب أن يجتاز البناء (`pnpm build`) بنجاح بدون أخطاء أو تحذيرات حرجية
- يجب أن يجتاز فحص الكود (`pnpm lint`) بدون أخطاء
- يجب أن يحصل على موافقة (Approval) من مطور واحد على الأقل غير منشئ الطلب
- يجب أن تكون جميع التعليقات (Comments) في طلب السحب محلولة (Resolved)
- لا يجب أن يوجد تعارضات (Merge Conflicts) مع فرع `main`
- يجب أن يكون الالتزام مُنسَقًا وفقًا لاتفاقية Conventional Commits

⚠ حجم طلب السحب

يُفضل أن يكون كل طلب سحب مركّزًا على مهمة واحدة. إذا تجاوز الطلب 400 سطر من التغييرات (باستثناء الملفات المُولّدة)، يجب تقسيم طلبات أصغر.

مراجعة الكود (Code Review)

مراجعة الكود هي حجر الزاوية في ضمان جودة وسلامة مشروع فيكسور. الهدف من المراجعة ليس فقط اكتشاف الأخطاء البرمجية، بل نشر المعرفة بين أعضاء الفريق، وضمان التوافق مع المعايير المعمول بها، والحفاظ على اتساق البنية البرمجية. كل كود يُدمج في أن يكون قد مر بمراجعة صريحة من مطور آخر.

مبادئ المراجعة

- المراجعة البناءة:** التركيز على تحسين الكود وليس نقد المطور. استخدام لغة محترمة ومحددة في التعليقات

- **السرعة:** الرد على طلبات المراجعة خلال 24 ساعة كحد أقصى. لا تترك الطلبات معلقة لأيام
- **الشمولية:** مراجعة المنطق والأمان والأداء وقابلية الصيانة، وليس فقط بناء الجملة
- **التسلسل الهرمي:** التعليقات على النمط (Style) اختيارية، بينما التعليقات على المنطق والأمان إلزامية المعالجة

نقاط التركيز في المراجعة

الناحية	ما يتم فحصه
الأمان	صلاحيات RLS، حماية <code>service-role</code> ، عدم تسريب بيانات حساسة للعميل
الأداء	استعلامات غير ضرورية، تسريبات ذاكرة، أحجام حزم كبيرة
الأنماط	التزام بنمط الخوادم فقط (<code>.server.ts</code>)، استخدام <code>useStableServerFn</code>
معالجة الأخطاء	استخدام <code>RouteErrorBoundary</code> ، تغليف الاستدعاءات بمعالجة أخطاء
النوعية	غياب <code>any</code> ، أنواع TypeScript صحيحة، تغطية اختبارية كافية

✓ قواعد الموافقة

موافقة واحدة كافية للدمج. التعليقات المُعلَّمة كـ **blocking** يجب معالجتها قبل الدمج. المُراجع يمكنه طلب تعديلات (Request Changes) الموافقة (Approve) أو التعليق فقط (Comment).

06

الاختبار (Testing)

يعتمد مشروع فيكسور على طبقتين أساسيتين للاختبار: **Vitest** للاختبارات الوحدوية (Unit Tests) التي تتحقق من صحة البرمجي المعزول، و**Storybook** (مع `@chromatic-com/storybook`) لاختبار المكونات المرئية بشكل منفصل عن هاتان الطبقتان معًا توفران شبكة أمان قوية تمنع الانحدارات (Regressions) وتضمن استقرار الواجهة.

اختبارات الوحدوية بـ Vitest

تُستخدم لاختبار الدوال المساعدة (Utility Functions)، ومُحوّلات البيانات (Data Transformers)، والمنطق البرمجي البحت. الاختبارات سريعة ومحددة وقابلة للتكرار. كل اختبار يجب أن يكون مستقلًا عن غيره ولا يعتمد على حالة مشتركة أو ترتيب تنفيذ معين.

```
نمط اختبار معتمد في فيكسور
import { formatPrice } from './utils'

describe('formatPrice', () => {
  it('should format positive numbers with commas', () => {
    expect(formatPrice(1234567.89)).toBe('1,234,567.89')
  })

  it('should handle zero correctly', () => {
    expect(formatPrice(0)).toBe('0.00')
  })

  it('should handle negative numbers', () => {
    expect(formatPrice(-500)).toBe('-500.00')
  })
})
```

اختبار المكونات بـ Storybook

يوفر Storybook بيئة معزولة لتطوير واختبار المكونات المرئية بشكل مستقل. من خلال التكامل مع Chromatic، يتم التقاط (Snapshots) تلقائيًا لكل نسخة ومقارنتها بالإصدار السابق لضمان عدم حدوث تغييرات غير مقصودة في المظهر. يُطلب كتابة واحدة على الأقل لكل مكون UI جديد.

Storybook + Chromatic

اختبارات مرئية للمكونات مع مقارنة لقطات الشاشة تلقائيًا
Chromatic لكل طلب سحب.

Vitest

اختبارات وحدوية سريعة للمنطق والدوال المساعدة. ملفات الاختبار
بجانب الملف المصدر أو في مجلد __tests__.

⚠ مبدء الاختبار

لا يُطلب تغطية اختبارية 100%، لكن يجب اختبار كل مسار خطأ حرج وكل دالة معقدة تحوي شروطًا متعددة. الأولوية للمنطق المالي والأمان
وصلاحيات الوصول.

07

التكامل والنشر المستمر (CI/CD)

تعتمد فيكسور على منصة **Vercel** كنظام أساسي للنشر المستمر (Continuous Deployment). عملية النشر تتم تلقائيًا عند كل **main**، حيث يقوم Vercel ببناء المشروع ونشره دون تدخل يدوي. هذا يضمن وصول أحدث التعديلات إلى المستخدمين بأسرع
مع الحفاظ على جودة عالية عبر عمليات الفحص التلقائية.

خطوة البناء (Build Process)

تبدأ عملية النشر بتنفيذ أمر **pnpm build** الذي يعمل على مرحلتين متسلسلتين. المرحلة الأولى هي **vite build** الذي
التطبيق وتحسينه للإنتاج. المرحلة الثانية هي تنفيذ **scripts/fix-vercel-bundle.mjs**، وهو سكريبت مخصص يع
محددة تتعلق بحزم النشر على بيئة Vercel، بما في ذلك ضمان توافق مسارات الوحدات مع نظام Nitro المستخدم في **ck Start**

التكوين التقني للبناء

- **Vite 7.3.1** — أداة البناء الرئيسية مع دعم Lightning CSS لتحسين الأنماط
- **TanStack Start 1.168.25** — إطار العمل مع دعم التوجيه المبني على الملفات وتقديم الخادم (SSR)
- **chunkSizeWarningLimit: 700KB** — حد تحذير لحجم الحزمة تم ضبطه وفقًا لاحتياجات المشروع
- **CSP Headers** — رؤوس أمان المحتوى مُهيأة على مستوى الخادم
- **frame-ancestors** — مُهيأ خصيصًا للسماح بتضمين التطبيق داخل Telegram

🔧 سكريبت fix-vercel-bundle

سكريبت **scripts/fix-vercel-bundle.mjs** يعالج تعقيدات نشر تطبيقات TanStack Start على Vercel. يشمل ضبط مسارات
الاستيراد وتوافق وحدات Node.js مع بيئة Serverless الخاصة بـ Vercel. أي تعديل على هذا السكريبت يتطلب اختبار نشر كامل.

يتضمن خط أنابيب النشر أيضًا فحصًا تلقائيًا عبر ESLint أثناء البناء، مما يضمن عدم دمج كود يخالف قواعد المشروع. إذا فشل ا
الكود، يتوقف النشر تلقائيًا ويُبلَّغ الفريق عبر إشعارات GitHub.

08

عملية الإصدار (Release Process)

عملية الإصدار في فيكسور مبسطة ومباشرة بناءً على النشر المستمر عبر Vercel. بما أن النشر يتم تلقائيًا من فرع **main**، فإن
يُمثل إصدارًا جديدًا فعليًا. ومع ذلك، تظل هناك حاجة لعملية إصدار منظمة للإصدارات المهمة التي تتضمن تغييرات جوهرية
وتواصلًا مع المستخدمين.

سير عمل النشر

- 1 يُنشأ فرع الميزة من main ويُطوّر مع اختبارات كافية
- 2 يُرسل طلب سحب مع وصف كامل وقائمة مراجعة مكتملة
- 3 يُراجَع الكود من قبل مطور آخر وتُحلّ جميع التعليقات
- 4 يُدمج طلب السحب في main بعد الحصول على الموافقة
- 5 يبدأ Vercel عملية البناء تلقائيًا (Build + fix-vercel-bundle)
- 6 إذا نجح البناء، يُنفَّذ النشر إلى بيئة الإنتاج
- 7 يتم التحقق من التطبيق مباشرة بعد النشر عبر Sentry و Mixpanel

خطة التراجع (Rollback)

في حال اكتشاف مشكلة حرجية بعد النشر، توفر فيكسور طبقات حماية متعددة. الطبقة الأولى هي **Vercel Rollback** الذي يسم أي نشر سابق بنقرة واحدة من لوحة تحكم Vercel. الطبقة الثانية هي مراقبة الأخطاء عبر **Sentry** الذي يُبلِّغ فورًا عن أي زيادة غير معتادة في معدل الأخطاء. الطبقة الثالثة هي **Mixpanel** الذي يراقب مؤشرات أداء المستخدم ويكتشف التراجعات في الأداء.

التراجع

Vercel يتيح العودة للنشر السابق فورًا. Sentry يرصد الأخطاء والتدهورات تلقائيًا.

النشر التلقائي

كل دمج في main يُنشئ نشرًا تلقائيًا. لا يوجد نشر يدوي في المسار العادي.

إصدارات الطوارئ ⚠️

للإصلاحات العاجلة، يُسمح بالدمج المباشر بعد مراجعة واحدة سريعة مع إضافة HOTFIX في عنوان طلب السحب. يجب متابعة ذلك بمراجعة خلال 24 ساعة.

09

إدارة الإصدارات (Versioning)

تتبع فيكسور معيار **Semantic Versioning** (الإصدار الدلالي) لإدارة أرقام إصدارات المشروع. هذا المعيار يوفر طريقة واضحة للإشارة إلى طبيعة التغييرات في كل إصدار، ويساعد المستخدمين والمطورين على فهم تأثير التحديثات وتوافقها مع الإصدارات السابقة.

بنية رقم الإصدار

يتكون رقم الإصدار من ثلاثة أجزاء: **MAJOR.MINOR.PATCH** (رئيسي.فرعي.تصحيحي). كل جزء يُرفع وفقًا لنوع التغيير المُقدّم في القواعد ليست اختيارية — الالتزام بها ضروري لضمان توافق تبعات المشروع وتوقعات المستخدمين.

المستوى	متى يُرفع	التأثير	مثال
MAJOR (رئيسي)	تغييرات غير متوافقة مع الإصدارات السابقة (Breaking Changes)	يتطلب تعديل الكود المستهلك	
MINOR (فرعي)	إضافة ميزات جديدة متوافقة مع الإصدارات السابقة	إضافات بدون إزالة	
PATCH (تصحيحي)	إصلاح أخطاء بدون تغيير الواجهة البرمجية	لا تأثير على الواجهات	

feat: → رفع fix | MINOR: → رفع PATCH | التزامات BREAKING CHANGE: في التذييل → ترفع MAJOR. هذا التآزر بين الات

يُمْكِن من أتمتة تسجيل التغييرات.

الإصدارات المسبقة

للإصدارات التجريبية يُضاف لاحقة: alpha.1-2.1.0 أو beta.3-2.1.0 أو rc.1-2.1.0. تسلسل الأولوية: rc → beta → stable.

10

قواعد التوثيق (Documentation Rules)

التوثيق الجيد ليس رفاهية بل ضرورة هندسية في مشروع بحجم وتعقيد فيكسور. يُشترط توثيق كل واجهة برمجية عامة (API) مكون مُعاد استخدام، وكل منطق معقد يحتاج شرحًا. الهدف هو أن يتمكن أي مطور جديد من فهم قاعدة الكود دون الحاجة مطوّل.

JSDoc للدوال والمكونات

كل دالة عامة ومُصدّرة يجب أن تحتوي على تعليق JSDoc كامل يصف الغرض والمعاملات والقيمة المُرجعة. هذا لا يُساعد فقط يُحسّن أيضًا تجربة التطوير عبر إكمال تلقائي دقيق في محرر الكود.

Calculates the profit/loss percentage for a trade.

entryPrice - The price at which the position was opened
 exitPrice - The price at which the position was closed
 direction - Trade direction: 'long' or 'short'
 returns Profit/loss as a percentage (positive = profit)
 throws {Error} If entryPrice is zero or negative

Example:
 calcPnlPercent(50000, 55000, 'long') // returns 10

```
function calcPnlPercent(
  entryPrice: number,
  exitPrice: number,
  direction: 'long' | 'short'
): number { ... }
```

تعليقات الكود المصدري

- لماذا وليس ماذا: التعليقات يجب أن تشرح لماذا تم اتخاذ قرار معين، وليس ماذا يفعله الكود (الكود نفسه يجب أن يكون واضحًا).
- المنطق المعقد: أي منطق يحتوي على حسابات مالية أو شروط معقدة يجب توثيقه بتعليق يشرح السياق.
- التوافق (Workarounds): أي حل بديل لمشكلة في مكتبة خارجية يجب توثيقه مع إشارة للمشكلة الأصلية.
- TODO/FIXME: استخدام هذه العلامات مع رقم القضية: TODO(#123): add rate limiting

التوازن في التوثيق

تجنّب التعليقات الزائدة التي تعيد صياغة الكود. x++ // increment x تعليق ضار. الاستثناء الوحيد هو التعليقات التي تشرح لماذا. ماذا، المقياس: إذا حذفت التعليق، هل سيفهم المطور السبب؟

قواعد جودة الكود (Code Quality Rules)

تعتمد فيكسور على ثلاث طبقات مترابطة لضمان جودة الكود: **TypeScript في الوضع الصارم (Strict Mode)** كطبقة أولى الأنواع، و**ESLint** كطبقة ثانية لاكتشاف الأنماط السيئة والمخالفات، و**Prettier** كطبقة ثالثة لضمان تنسيق موحد عبر المشروع. الطبقات تعمل معًا لمنع فئات واسعة من الأخطاء قبل مرحلة التشغيل.

TypeScript — الوضع الصارم

المشروع مُهيأ بـ TypeScript 5.8.3 في الوضع الصارم (`strict: true`) مع دعم ES2022 ونظام حل الوحدات `tsconfig` `strictNullChecks` و `noImplicitAny` و `strictIndexSignature` استخدام النوع `any` ممنوع إلا في حالات استثنائية مُوثَّقة بتعليق يشرح السبب.

ESLint — فحص الكود

يتم تشغيل ESLint عبر `pnpm lint` على المجلدات `src/` و `server/`. فحص ESLint يكشف مشاكل مثل الـ `any` المستخدمة، والوصول إلى خصائص قد تكون غير معرَّفة، والأنماط المعرضة للأخطاء. يجب أن يجتاز الكود فحص ESLint بدمج.

Prettier — التنسيق التلقائي

يتم تشغيل Prettier عبر `pnpm format` لتنسيق جميع الملفات تلقائيًا. هذا يُلغي النقاشات حول التنسيق ويضمن مظهرًا متسقًا للمحرر لتنفيذ Prettier عند الحفظ (Format on Save) لتقليل الالتزامات التنسيقية.

الأداة	الأمر	النطاق	الدور
TypeScript	بناء تلقائي	المشروع بالكامل	فحص الأنواع في وقت التحويل البرمجي
ESLint	<code>pnpm lint</code>	<code>src/</code> <code>server/</code>	اكتشاف الأنماط السيئة والمخالفات
Prettier	<code>pnpm format</code>	المشروع بالكامل	تنسيق موحد للكود والأنماط

✓ حماية الخادم

نظام حماية الطبقات: `server.ts` يمنع الاستيراد من جهة العميل عبر TanStack Start، وطبقة الأمان `security.ts` تحمي جميع النهاية الـ 14، و `useRenderGuard` يحمي من التنفيذ غير الصحيح.

قواعد الاعتماديات (Dependency Rules)

إدارة الاعتماديات في فيكسور تخضع لقواعد صارمة لحماية المشروع من التضخم في حجم الحزم، ومن الثغرات الأمنية، ومن المكتبات. كل إضافة اعتمادية جديدة يجب أن تكون مُبرَّرة ومدروسة، مع تفضيل الحلول المبنية على المتصفح أو المكتبات المشروعة.

قواعد إضافة الحزم

- **التقييم المسبق:** قبل إضافة أي حزمة، يجب البحث عن بديل مُدمج في المشروع أو في مكتبة قياسية.
- **حجم الحزمة:** مراجعة حجم الحزمة عبر `bundlaphobia.com` — لا تقبل الحزم التي تزيد حزمة التطبيق بشكل كبير.
- **الصيانة النشطة:** تجنب الحزم التي لم تتلق تحديثًا خلال الأشهر الستة الماضية.
- **الترخيص:** التأكد من توافق ترخيص الحزمة مع ترخيص المشروع.
- **الأمان:** فحص الحزمة عبر `pnpm audit` قبل وبعد الإضافة.

مكتبة **CCXT** (للتواصل مع بورصات العملات الرقمية) تُمثّل حالة خاصة في فيكسور. نظرًا لحجمها الكبير ومتطلباتها البيئية، تمّ (Externalized) بحيث تعمل على الخادم فقط ولا تُضمّن أبدًا في حزمة العميل (Client Bundle). هذا يحقق هدفين: حماية الحساسية من التسريب للعميل، وتقليل حجم الحزمة المُرسلة للمتصفح بشكل كبير.

```
server.ts: صحيح - في م
cxt from 'ccxt'
```

لا يمكن استيراده من الملفات العادية CCXT - >
TanStack Start سيتم حطره بواسطة

أنواع الاعتماديات

dependencies: حزم مطلوبة في بيئة الإنتاج (React, TanStack). **devDependencies**: حزم للتطوير فقط (ESLint, Prettier, Vitest). أداة تطوير كاعتمادية إنتاج أبدًا.

تحديث الاعتماديات

التحديثات يجب أن تكون في طلب سحب منفصل (`chore(deps)` : ...). لا تخط تحديث الاعتماديات مع تغييرات الميزات. اختبر بالكامل بعد كل تحديث رئيسي (Major).

13

قائمة مراجعة المراجعة (Review Checklist)

القائمة الشاملة التالية تُستخدم كمرجع موحد لكل مراجعة كود في فيكسور. تضمن هذه القائمة عدم تفويت أي جانب مهم أثناء المراجعة. يُنصح بالرجوع إليها قبل بدء المراجعة وبعدها للتأكد من الشمولية.

الأمان والحماية

- لا توجد مفاتيح API أو أسرار في الكود المصدري أو في حزمة العميل
- صلاحيات **Supabase RLS** مُهيأة بشكل صحيح لجميع الجداول المتأثرة
- مفتاح **service-role** يُستخدم في ملفات **server.ts**. فقط
- CCXT ومكتبات الخادم الثقيلة لا تُستورد من جهة العميل
- رؤوس CSP مُهيأة ولا تمنع الموارد الشرعية
- لا يوجد حقن SQL أو XSS — جميع المدخلات مُحققة ومُهروبة

النوعية والأداء

- لا يوجد استخدام لـ **any** بدون تعليق مُبرّر
- جميع الدوال والمتغيرات لها أنواع صريحة حيثما يكون ذلك ضروريًا
- لا توجد استعلامات قاعدة بيانات غير ضرورية أو حلقات غير محسّنة
- حجم الحزم لا يتجاوز حدود التحذير (**chunkSizeWarningLimit: 700KB**)
- مكونات React لا تعيد التحويل (Re-render) بدون سبب — استخدام **useMemo** و **useCallback** عند الحاجة

البنية والأنماط

- الملفات الخادمية تستخدم لاحقة **server.ts**. وتُحمّل عبر **useStableServerFn**
- معالجة الأخطاء موجودة: **RouteErrorBoundary** على المستوى المناسب
- المكونات الجديدة تحتوي قصص Storybook عند الحاجة

- الالتزامات تتبع اتفاقية Conventional Commits
- لا يوجد كود مكرر (DRY) — المنطق المشترك مُستخرج إلى دوال مشتركة

التوثيق والاختبار

- الدوال العامة تحتوي تعليقات JSDoc كاملة
- المنطق المالي والمُعقّد مُوثّق بشرح السياق
- الاختبارات تغطي المسارات الحرجة والأخطاء المتوقعة
- لا توجد تعليقات `console.log` في الكود المُقدّم
- المسارات الجديدة تحتوي على معالجة أخطاء مناسبة

✓ التأكيد النهائي

قبل الموافقة، تأكد أن: **البناء ينجح**، **ESLint نظيف**، **لا تعارضات**، **جميع التعليقات مُحلّة**، **لا مخاطر أمنية**. إذا استوفى طلب السحب هذا، فالموافقة مبررة.

وثيقة رسمية لمنظومة فيكسور الهندسية

VIXOR — AI-Powered Trading Terminal